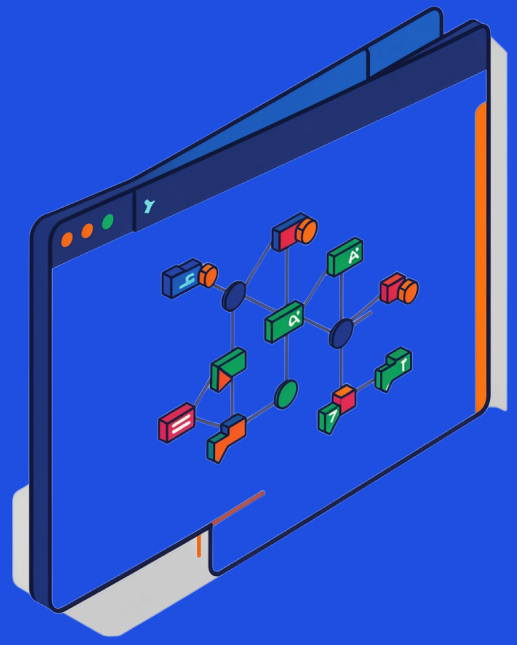
FINAL REVIEW HACKATHON

Zombie and Orphan Process Detection System



Operating System and System Programming (25CS2104E)

K. Samanth — Roll No: 2520080011
P. Sai Suraj — Roll No: 2520090179
V. Pavan Kumar — Roll No: 2520090079

Under the Esteemed Guidance of: Raghupathi M, CSE Department

Introduction



Normal lifecycle

A process is a program currently being executed; each process has a PID and normally a PPID. Parents create children with **fork()**; relationships shape process lifecycle and cleanup.



Zombie

A zombie has terminated but remains in the process table until its exit status is collected.



Orphan

An orphan is a still-living child whose parent terminates first; Linux reparents it.

OBJECTIVE

Make process state visible

Identify zombie and orphan candidates, show PID/PPID/state, and reveal parent-child relationships.

Distinct labels keep “terminated” and “still living” separate.

Problem Statement and Abstract

Manual identification is difficult when many processes are active.

01

Problem

Unreaped children can accumulate as zombies; early parent termination can create orphan conditions.

02

Detection

The tool inspects Linux process metadata and detects potential conditions.

03

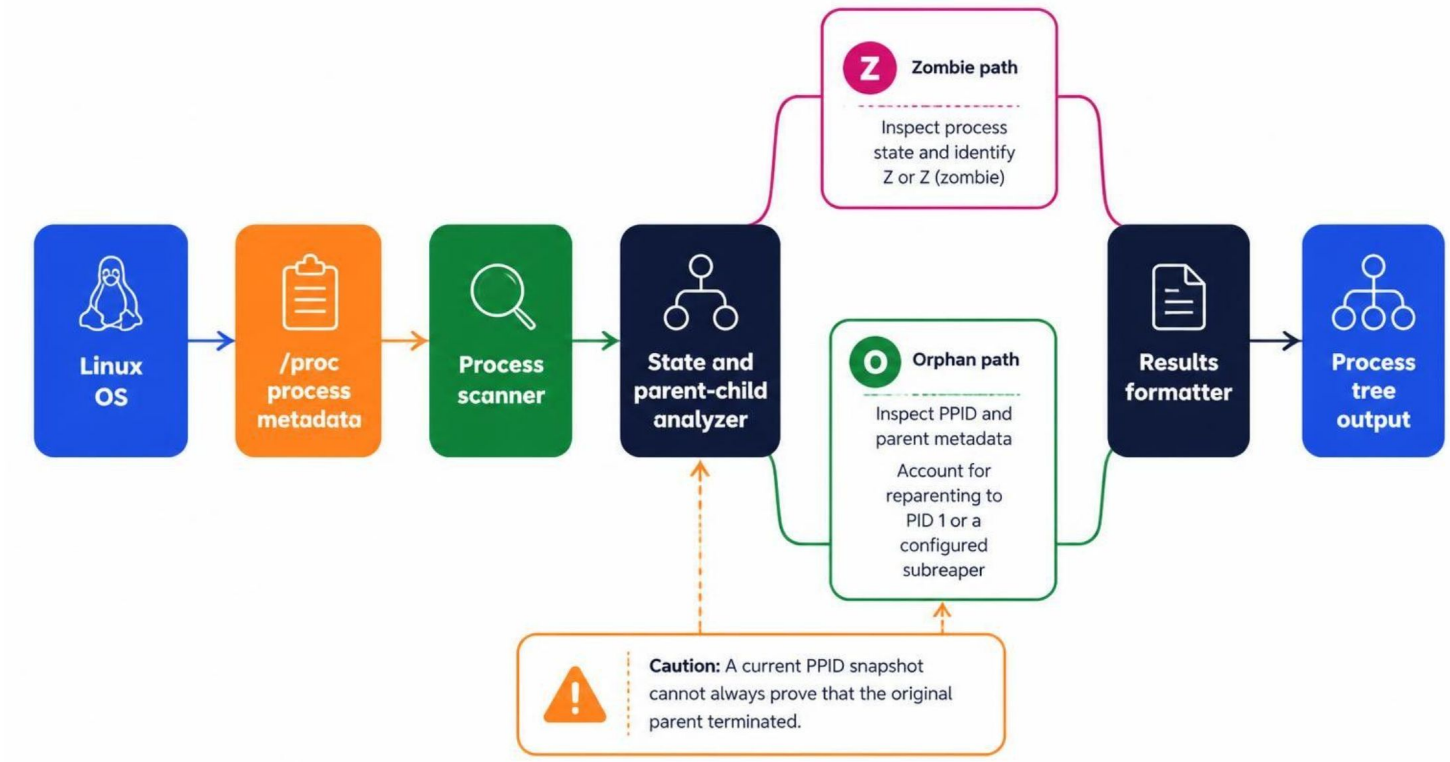
Report

Parentage, PID, PPID, name, and state are presented clearly.

Key objectives: detect state **Z**, identify orphan candidates, display PID/PPID/name/state, and support OS learning and basic debugging.

System Design and Architecture

A single metadata pipeline separates zombie evidence from orphan evidence.



Technologies and Requirements

Area	Requirements
Software	Linux/Ubuntu, C, GCC, Ubuntu Terminal, Linux **/proc**.
Headers	stdio.h, stdlib.h, string.h, dirent.h, unistd.h, and implementation-specific headers.
Interfaces	fork(), getpid(), getppid(), wait(), waitpid(), /proc/[PID]/stat, opendir(), readdir().
Hardware	standard Ubuntu/Linux computer; 4 GB RAM recommended for the lab environment.

fork() and waitpid() support controlled experiments; the monitor inspects existing metadata through /proc.

Implementation Approach

Five compact stages turn /proc entries into a readable relationship report.

1
Enumerate

Scan numeric directories under /proc.

2
Extract

Retrieve PID, PPID, process name, and state.

3
Detect

Report state Z; analyze parent metadata for supported orphan evidence.

4
Relate

Organize parent-child relationships into a hierarchy.

5
Report

Print readable process details and relationship output.

C-STYLE PSEUDOCODE

For each process in /proc: read PID / PPID / name / state; if state is Z, report a zombie; otherwise analyze parent metadata and report an orphan candidate only when evidence supports it; display the relationship.

Safeguards

/proc can change during scanning. Handle missing entries and permission errors safely.

Detection Logic and Process Relationships

The same parent-child graph can show three materially different states.

NORMAL

Parent is alive

Child may run or terminate and be reaped normally.

ZOMBIE

Terminated; awaiting reaping

State is **Z**; execution ended but the process-table entry remains until reaping.

ORPHAN

Living; reparented

Original parent ends first; a living child is reparented to PID 1 or another subreaper.

Illustrative tree

Before termination: Parent P (PID 1000) → Child A (PID 1001, running) and Child B (PID 1002, zombie).

After termination: Child A is shown under an illustrative reaper PID.

Key difference

Zombie = terminated and awaiting reaping.
Orphan = living child whose original parent terminated.

PIDs are illustrative, not measured results.

Testing and Validation

Expected behaviors define the test plan; measurements and screenshots remain to be supplied.

Test case	Expected outcome
Normal process	Parent waits for child; no persistent zombie should remain after successful reaping.
Zombie	Child exits while parent delays wait(); it should temporarily appear as state Z.
Orphan	Parent terminates while child continues; observe reparenting.
Multiple processes	Verify distinct PIDs and parent-child relationships.
Permissions and races	Scan while processes start/exit; disappearing /proc entries must not crash the tool.

EVIDENCE RESERVED

Actual terminal screenshots and measured observations

Reserved area for supplied execution evidence.

No screenshots, success rates, benchmarks, or execution results are invented here.

Results, Impact, and Limitations

EXPECTED RESULTS

Detected zombie list with PID/PPID/state; orphan candidates; readable relationship tree; demonstration-ready terminal output.

IMPACT

Clarifies Linux process management, introductory debugging, lifecycle behavior, /proc metadata, and C programming.

LIMITATIONS

Orphans may be reparented before observation; current PPIDs cannot always reconstruct original parentage; process races, permissions, and distribution differences affect output.

Any mock output must be labeled illustrative rather than measured; use actual output screenshots only when supplied.

Conclusion and Thank You

The system separates zombie detection from orphan identification using Linux process states and parent-child relationships.

It demonstrates C-based inspection of Linux process interfaces for learning and basic debugging.

FUTURE ENHANCEMENTS

Graphical process-tree visualization · periodic monitoring · exportable reports · historical parent tracking · stronger handling of reparenting and transient states

Thank You

Made with GAMMA